SIMATS ENGINEERING

Engineer to Excel

SIMATS

Saveetha Institute of Medical And Technical Sciences

(Declared as Deemed to be University under Section 3 of UGC Act 1956)

Department of Computer Science and Engineering

Course Code & Name	CSA06 – Design and Analysis of Algorithms
Assessment	CO1 – Assessment Tool 1: IT Application-Based Problem Solving – Scalability Audit of an E-Commerce Platform
Course Outcome Assessed	Determine algorithm efficiency using asymptotic notations and mathematical techniques including Master Theorem and substitution method.
Duration	3 Days
Mode	Individual
Student Reg. No.	192424268
Student Name	J Rieona

Given Data and Assumptions

Before solving the individual parts, the data given in the problem and the assumptions used throughout the solution are listed below.

- **Catalogue size:** the current catalogue has $n = 100{,}000 = 10^5$ products and is expected to grow to $n = 1{,}000{,}000 = 10^6$ products within a year.
- **Server speed:** 10^8 basic operations per second. Therefore, running time in seconds = (number of basic operations) $\div 10^8$.
- **Nightly processing window:** 10 minutes = 600 seconds. The maximum number of operations that fit in the window is $600 \times 10^8 = 6 \times 10^{10}$ operations.
- **Logarithms:** all logarithms are to base 2. Useful values: $\log_2 100{,}000 \approx 16.61$ and $\log_2 1{,}000{,}000 \approx 19.93$.
- **Recurrences:** $T(1) = 1$ is used as the base case and n is taken as a power of 2 (the usual assumption for divide-and-conquer recurrences; the results extend to other values of n using floors and ceilings).
- **Notation:** $O(g(n))$ is an asymptotic upper bound, $\Omega(g(n))$ is an asymptotic lower bound and $\Theta(g(n))$ is a tight bound (both O and Ω).

Part (a): Understand the Problem

The table below identifies, for every module, the input size, the basic operation and the expected growth of the number of executions. The justification for each row follows the table.

Module	Input size	Basic operation	Expected growth of its execution count
M1	n = number of products in the catalogue (Product[0 … n−1])	Comparison of Product[i].title with Product[j].title (and seller) in the IF statement of the inner loop	About $n^2/2$ comparisons. Quadratic: 10 times more products gives about 100 times more work.
M2	n = upper limit of the request window (loop runs while i < n)	Audit(Request[j]) in the inner FOR loop	1 + 2 + 4 + … which is less than 2n. Linear: 10 times larger n gives about 10 times more work.
M3 – S1	n = number of entries in the unsorted catalogue	Comparison of the search key with a catalogue entry	Between 1 and n comparisons. Linear in n.
M3 – S2	n = number of entries in the sorted index	Comparison of the key with the middle entry of the current index range	$\log_2 n + 1$ comparisons. Logarithmic: doubling n adds only one comparison.
M4	n = size of the customer–product data	Merge step: combining the partial results (the f(n) = n work done after the recursive calls)	Current design about n^2 (quadratic). Redesign about n log n.
M5	n = number of sales records	Comparison of two records by revenue during sorting at each recursion level. For the report generator, the operation counted by the $3n^2$ term	Analytics engine about $n \log^2 n$. Report generator about n^2 (quadratic).

Module M1 – Duplicate-Listing Detector

Input size. The routine works on the array of n products, so the input size is n.

Basic operation and where it occurs. The statement that is executed most often is the IF test inside the inner loop, which compares the title and the seller of product i with those of product j. It is executed once for every pair (i, j) with i < j. The call MarkDuplicate(j) is executed only

when a duplicate is actually found, so it depends on the data and not on the loop structure; it is a constant-time action and cannot be executed more often than the comparison that guards it.

Expected growth. Every product is compared with every later product, so the number of comparisons is the number of unordered pairs, $n(n-1)/2 \approx n^2/2$. If n is multiplied by 10, the work is multiplied by about 100; if n doubles, the work becomes about four times larger.

Module M2 – API Rate Limiter

Input size. The input size is n, the upper limit up to which the window size i is allowed to grow (the number of requests that can be audited).

Basic operation and where it occurs. The operation Audit(Request[j]) is marked as the basic operation in the pseudocode. It is the body of the inner FOR loop, so it is executed once for every value of j in every pass of the WHILE loop.

Expected growth. The inner loop does not run n times. In the pass in which the window size is i, it runs exactly i times, and i doubles after every pass (1, 2, 4, 8, …). The total is therefore a geometric series $1 + 2 + 4 + \ldots + 2^{k-1}$ whose value is dominated by its last term and stays below 2n. The count grows linearly with n.

Module M3 – Product Search (versions S1 and S2)

Input size. For S1 the input size is n, the number of products in the unsorted catalogue. For S2 it is n, the number of entries in the sorted product index.

Basic operation and where it occurs. In both versions the basic operation is the comparison of the search key with an entry. In S1 it occurs once for every entry scanned in the sequential loop, so up to n times. In S2 it occurs once per halving step, when the key is compared with the middle entry.

Expected growth. S1 grows linearly with n (10 times more products means up to 10 times more comparisons). S2 satisfies $T(n) = T(n/2) + 1$, so it needs about $\log_2 n + 1$ comparisons; multiplying n by 10 adds only about $\log_2 10 \approx 3.3$ comparisons.

Module M4 – Recommendation Engine

Input size. The input size is n, the size of the customer–product data.

Basic operation and where it occurs. The dominant work done at each recursive call, apart from the recursive calls themselves, is the linear-time merge of the partial results. It is executed

after the sub-problems return, at every node of the recursion tree, and contributes the term n in $T(n) = 4T(n/2) + n$.

Expected growth. In the current design the number of sub-problems quadruples at every level while their size only halves, so the total work grows like n^2 (10 times more data gives about 100 times more work). In the redesign, two sub-problems of half size are created at each level, so every level costs n and there are $\log_2 n$ levels; the work grows like n log n (10 times more data gives about 12 times more work).

Module M5 – Sales Analytics Engine and Report Generator

Input size. The input size is n, the number of sales records.

Basic operation and where it occurs. In the analytics engine, the basic operation is the comparison of two records by revenue during the sort carried out at every level of the recursion; sorting a part of size m costs about m log m comparisons, which gives the term n log n in $T(n) = 2T(n/2) + n \log n$. For the report generator, profiling counts the total operations as $f(n) = 3n^2 + 10n \log_2 n + 50$. The dominant operation is the one counted by the $3n^2$ term, because the other two terms grow more slowly.

Expected growth. The analytics engine grows like $n \log^2 n$ (10 times more records gives about 14 times more work). The report generator grows quadratically (10 times more records gives about 100 times more work).

Part (b): Analyse the Iterative Modules

Module M1 – exact number of comparisons

Line	Statement	Explanation
1	FOR i ← 0 TO n − 2 DO	Outer loop. The variable i is the index of the "first" product of a pair. It takes the values 0, 1, …, n − 2, so the loop runs n − 1 times. It stops at n − 2 because the last product (index n − 1) has no later product to be compared with.

Line	Statement	Explanation
2	FOR j ← i + 1 TO n − 1 DO	Inner loop. The variable j runs over all products after product i. For a fixed i, j takes the values i + 1, …, n − 1, which is (n − 1) − (i + 1) + 1 = n − 1 − i values. Starting at i + 1 ensures that a product is never compared with itself or with an earlier product, so every unordered pair is examined exactly once.
3	IF Product[i].title = Product[j].title AND Product[i].seller = Product[j].seller THEN	Basic operation. The titles and sellers of the two products are compared. This statement is executed once in every iteration of the inner loop. (Because AND stops early when the title differs, the seller is checked only for equal titles, but the pair test is counted as one comparison.)
4	MarkDuplicate(j)	Executed only when both conditions are true, i.e. when a real duplicate exists. It takes constant time and does not change the number of comparisons.
5	END IF	End of the IF block. No cost.
6	END FOR	End of the inner loop.
7	END FOR	End of the outer loop.

Derivation of the exact count.

Step 1: For a fixed value of i, the inner loop executes the comparison once for each j from i + 1 to n − 1:

$$\text{Comparisons for a given } i = (n - 1) - (i + 1) + 1 = n - 1 - i$$

Step 2: Add this over all values of the outer loop, i = 0 to n − 2:

$$C(n) = \Sigma_{i=0}^{n-2} (n - 1 - i)$$

Step 3: Substitute k = n − 1 − i. When i = 0, k = n − 1, and when i = n − 2, k = 1. The sum becomes the sum of the first n − 1 natural numbers:

$$C(n) = (n - 1) + (n - 2) + \ldots + 2 + 1 = \Sigma_{k=1}^{n-1} k$$

Step 4: Use the formula $\Sigma_{k=1}^{m} k = m(m + 1)/2$ with m = n − 1:

$$C(n) = (n - 1)(n - 1 + 1)/2 = n(n - 1)/2 = (n^2 - n)/2$$

Check: for n = 4 the pairs are (0,1), (0,2), (0,3), (1,2), (1,3), (2,3), which is 6 comparisons, and 4 × 3/2 = 6.

Time complexity. $C(n) = n^2/2 - n/2$. For $n \geq 2$ we have $n^2/4 \leq (n^2 - n)/2 \leq n^2/2$ (the left inequality is equivalent to $n^2 \geq 2n$, which holds for $n \geq 2$). Hence $C(n) = \Theta(n^2)$. The loops do not depend on the data and there is no early exit, so all $n(n - 1)/2$ comparisons are always performed. The best, average and worst cases are all $\Theta(n^2)$.

Space complexity. The algorithm needs only the loop counters i and j and a constant amount of extra memory for the flag set by MarkDuplicate. The auxiliary space is $\Theta(1)$. (The input itself occupies $\Theta(n)$ memory, but this is not counted as extra space.)

Meaning for ShopSphere. For n = 100,000 the routine performs 4,999,950,000 comparisons (about 50 seconds). For n = 1,000,000 it performs 499,999,500,000 comparisons (about 5,000 seconds, i.e. 83 minutes).

Module M2 – exact number of Audit operations

Line	Statement	Explanation
1	i ← 1	Initialises the window size to 1. Executed once.
2	WHILE i < n DO	Outer loop. It continues as long as the window size i is smaller than n. Since i doubles every time, the values of i are 1, 2, 4, 8, … and the loop stops as soon as i ≥ n.
3	FOR j ← 0 TO i − 1 DO	Inner loop. For the current window size i, j takes the values 0, 1, …, i − 1, so the loop body runs exactly i times (not n times).
4	Audit(Request[j])	Basic operation. Executed once for each j, i.e. i times in the current pass of the WHILE loop.
5	END FOR	End of the inner loop.
6	i ← i × 2	Doubles the window size. After m passes, $i = 2^m$.
7	END WHILE	End of the outer loop.

Derivation of the exact count.

Step 1: In the pass number m (m = 0, 1, 2, …) the value of i is 2^m. The loop continues while $2^m < n$, that is, while $m < \log_2 n$.

Step 2: Number of passes of the WHILE loop. If $n = 2^k$, the passes are $m = 0, 1, \ldots, k - 1$, so there are $k = \log_2 n$ passes. For a general n there are $\lceil \log_2 n \rceil$ passes.

Step 3: In pass m the inner loop runs $i = 2^m$ times, so pass m performs 2^m Audit operations.

Step 4: Add the work of all passes:

$$A(n) = \Sigma_{m=0}^{k-1} 2^m = 2^0 + 2^1 + 2^2 + \ldots + 2^{k-1}$$

Step 5: This is a geometric series. Using $\Sigma_{m=0}^{k-1} r^m = (r^k - 1)/(r - 1)$ with $r = 2$:

$$A(n) = (2^k - 1)/(2 - 1) = 2^k - 1$$

Step 6: Since $n = 2^k$, we get the exact count

$$A(n) = n - 1 \quad \text{(for n a power of 2)}$$

For a general n, with $k = \lceil \log_2 n \rceil$, $A(n) = 2^k - 1$. Because $n \leq 2^k < 2n$, we have $n - 1 \leq A(n) < 2n - 1$. In every case, A(n) lies between n − 1 and 2n, so $A(n) = \Theta(n)$.

Is the team member's claim of Θ(n log n) correct? No. The claim assumes that the inner loop runs n times in each of the log n passes, which would give n log n. In reality, the inner loop runs i times, and i is small in the early passes (1, 2, 4, …). Only the last pass is close to n, and the passes before it add up to about the same amount again ($1 + 2 + \ldots + n/2 = n - 1$). Formally, $A(n)/(n \log_2 n) = (n - 1)/(n \log_2 n) \to 0$ as $n \to \infty$. So A(n) grows strictly slower than n log n, and A(n) is not Ω(n log n). The statement A(n) = O(n log n) is true but it is not a tight bound.

Tight bound: $A(n) = \Theta(n)$. The time complexity is Θ(n) and the space complexity is Θ(1), because only i and j are used as extra variables.

Verification for n = 1,024 (= 2^{10}). The WHILE loop runs while $i < 1024$. The value $i = 1024$ fails the test, so the passes are $i = 1, 2, 4, \ldots, 512$, which is 10 passes (= $\log_2 1024$).

Pass number (m)	Window size i	Audit operations in the pass	Running total
0	1	1	1
1	2	2	3
2	4	4	7
3	8	8	15

Pass number (m)	Window size i	Audit operations in the pass	Running total
4	16	16	31
5	32	32	63
6	64	64	127
7	128	128	255
8	256	256	511
9	512	512	1,023

The total is $1 + 2 + 4 + \ldots + 512 = 1{,}023$. The formula gives $2^{10} - 1 = 1{,}023 = n - 1$, so the derivation is verified. The team member's formula $n \log_2 n$ would give $1{,}024 \times 10 = 10{,}240$ operations, which is about ten times too large. Note also that the last pass alone (512 operations) accounts for about half of all the work, which confirms that the cost is linear.

Part (c): Solve a Recurrence by Substitution

Recurrence: $T(n) = T(n/2) + 1$ with $T(1) = 1$. Assume $n = 2^k$ so that $n/2$ is always an integer.

Meaning of each part. The term +1 is the single comparison of the key with the middle entry. The term $T(n/2)$ is the cost of continuing the search in the one half that remains (the other half is discarded). The base case $T(1) = 1$ says that when only one entry is left, one comparison decides the result.

Backward substitution.

Step 1: Write the recurrence for n:

$$T(n) = T(n/2) + 1$$

Step 2: Replace $T(n/2)$ by its own recurrence, $T(n/2) = T(n/4) + 1$:

$$T(n) = [T(n/4) + 1] + 1 = T(n/4) + 2$$

Step 3: Replace $T(n/4)$ by $T(n/8) + 1$:

$$T(n) = [T(n/8) + 1] + 2 = T(n/8) + 3$$

Step 4: Replace $T(n/8)$ by $T(n/16) + 1$:

$$T(n) = T(n/16) + 4$$

Step 5: Observing the pattern, after k substitutions:

$$T(n) = T(n/2^k) + k$$

Step 6: The substitution stops at the base case, $n/2^k = 1$, i.e. $2^k = n$, i.e. $k = \log_2 n$.

Step 7: Substitute $k = \log_2 n$ and $T(1) = 1$:

$$T(n) = T(1) + \log_2 n = 1 + \log_2 n$$

Exact closed form: $T(n) = \log_2 n + 1$ for n a power of 2, and $T(n) = \lfloor \log_2 n \rfloor + 1$ for any $n \geq 1$.

Verification by induction. Base case: $T(1) = 1$ and $\log_2 1 + 1 = 1$. Inductive step: assume $T(n/2) = \log_2(n/2) + 1 = \log_2 n$. Then $T(n) = T(n/2) + 1 = \log_2 n + 1$, which is the formula. Numerical check: $T(2) = T(1) + 1 = 2$, $T(4) = 3$, $T(8) = 4$, and the formula gives $1 + 1$, $2 + 1$, $3 + 1$ respectively.

Meaning for ShopSphere. For n = 1,000,000 the search needs $\lfloor 19.93 \rfloor + 1 = 20$ comparisons at most.

Time and space complexity of the two implementations.

Implementation	Time complexity	Space complexity	Reason
Iterative	$\Theta(\log n)$	$\Theta(1)$	A loop with the variables low, high, mid and key halves the range each time, so there are at most $\log_2 n + 1$ iterations. Only a constant number of variables is needed.
Recursive	$\Theta(\log n)$	$\Theta(\log n)$	The number of comparisons is the same, but every recursive call adds a stack frame. The recursion depth is $\lfloor \log_2 n \rfloor + 1$, so the call stack needs $\Theta(\log n)$ space (this assumes that the index range is passed by indices and the array is not copied; copying the half would cost $\Theta(n)$).

Part (d): Apply the Master Theorem

Master Theorem. For $T(n) = aT(n/b) + f(n)$ with $a \geq 1$ and $b > 1$, compare f(n) with n^(log_b a):

- **Case 1:** if f(n) = O(n^(log_b a − ε)) for some ε > 0, then T(n) = Θ(n^(log_b a)). The leaves of the recursion tree dominate.
- **Case 2:** if f(n) = Θ(n^(log_b a)), then T(n) = Θ(n^(log_b a) log n). Every level costs the same.
- **Case 3:** if f(n) = Ω(n^(log_b a + ε)) for some ε > 0 and a·f(n/b) ≤ c·f(n) for some constant c < 1 (regularity condition), then T(n) = Θ(f(n)). The root dominates.
- **Extended Case 2:** if f(n) = Θ(n^(log_b a) $\log^k$ n) with k ≥ 0, then T(n) = Θ(n^(log_b a) $\log^{k+1}$ n).

Module M4 – current design: T(n) = 4T(n/2) + n

Step 1 – identify the parameters: a = 4, b = 2, f(n) = n.

Step 2 – compute n^(log_b a):

$$n^{(\log_2 4)} = n^2$$

Step 3 – compare f(n) with n²: f(n) = n = n^{2-1}, so f(n) = O($n^{2-\varepsilon}$) with ε = 1 > 0. The merge work is polynomially smaller than n².

Step 4 – case: Case 1 applies. Therefore

$$T(n) = \Theta(n^2)$$

Exact solution (needed for part g). By backward substitution: T(n) = 4T(n/2) + n = 16T(n/4) + 2n + n = 64T(n/8) + 4n + 2n + n = … = 4^k T(n/2^k) + (2^k − 1)n. With 2^k = n and T(1) = 1, we get T(n) = n² + (n − 1)n = 2n² − n.

Effect of doubling the data. T(2n) ≈ 2(2n)² = 8n², while T(n) ≈ 2n². The running time becomes about **4 times** larger each time the data doubles. For example, at n = 1,000,000 the work is about 2 × 10^{12} operations (5.6 hours), and for 2,000,000 products it would be about 8 × 10^{12} operations (about 22 hours).

Module M4 – proposed redesign: T(n) = 2T(n/2) + n

Step 1 – identify the parameters: a = 2, b = 2, f(n) = n.

Step 2 – compute n^(log_b a):

$$n^{(\log_2 2)} = n^1 = n$$

Step 3 – compare f(n) with n: f(n) = n = Θ(n^(log_b a)). The two have the same growth rate.

Step 4 – case: Case 2 applies. Therefore

$$T(n) = \Theta(n \log n)$$

Exact solution. $T(n) = 2T(n/2) + n = 4T(n/4) + 2n = 8T(n/8) + 3n = \ldots = 2^k\, T(n/2^k) + kn$. With $2^k = n$, $k = \log_2 n$ and $T(1) = 1$: $T(n) = n \log_2 n + n$.

Effect of doubling the data. $T(2n)/T(n) \approx 2n \log(2n)/(n \log n) = 2(1 + 1/\log_2 n)$. For n = 1,000,000 this is $2 \times (1 + 1/19.93) \approx 2.10$. The running time **slightly more than doubles**, which is far better than the factor of 4 of the current design.

Comparison. The redesign changes the complexity from $\Theta(n^2)$ to $\Theta(n \log n)$. Removing the replicated partitions reduces the number of sub-problems from four to two, so each level of the recursion tree costs n instead of growing by a factor of 2 at every level.

Module M5 – T(n) = 2T(n/2) + n log n

Step 1 – identify the parameters: a = 2, b = 2, f(n) = n log n.

Step 2 – compute n^(log_b a): $n^{(\log_2 2)} = n$.

Step 3 – test the three standard cases.

- **Case 1** needs $f(n) = O(n^{1-\varepsilon})$. This fails, because n log n grows faster than n.
- **Case 2** needs $f(n) = \Theta(n)$. This fails, because $f(n)/n = \log n \to \infty$, so f(n) is not bounded above by a constant multiple of n.
- **Case 3** needs $f(n) = \Omega(n^{1+\varepsilon})$ for some $\varepsilon > 0$. This fails, because $f(n)/n^{1+\varepsilon} = \log n / n^{\varepsilon} \to 0$ for every $\varepsilon > 0$ (a logarithm grows slower than any positive power of n).

Conclusion: f(n) = n log n is larger than n, but only by a logarithmic factor, so it falls in the gap between Case 2 and Case 3. **None of the three standard cases applies.**

Using the extended Master Theorem. Here $f(n) = \Theta(n^{(\log_b a)} \log^k n)$ with $n^{(\log_b a)} = n$ and $k = 1 \geq 0$. By the extended Case 2:

$$T(n) = \Theta(n \log^{1+1} n) = \Theta(n \log^2 n)$$

Verification using a recursion tree. Let $L = \log_2 n$.

- At level i (i = 0, 1, …, L − 1) there are 2^i nodes, each working on $n/2^i$ records and costing $(n/2^i) \log(n/2^i)$.
- The cost of level i is $2^i \times (n/2^i) \times \log(n/2^i) = n(\log n - i) = n(L - i)$.
- The leaves (level L) are n nodes costing T(1) = 1 each, so they cost n in total.

Adding all levels:

$$T(n) = n[L + (L - 1) + \ldots + 2 + 1] + n = n \cdot L(L + 1)/2 + n$$

$$T(n) = (n \log^2 n)/2 + (n \log n)/2 + n = \Theta(n \log^2 n)$$

Check: $T(2) = 2T(1) + 2 \log_2 2 = 4$ and the formula gives $2 \times 1 \times 2/2 + 2 = 4$. $T(4) = 2T(2) + 4 \times 2 = 16$ and the formula gives $4 \times 2 \times 3/2 + 4 = 16$. Both agree.

Meaning. Sorting again at every level of the recursion costs an extra log n factor compared with a plain merge-based design ($\Theta(n \log n)$). The analytics engine is acceptable for now but is a candidate for optimisation.

Part (e): Analyse Best, Average and Worst Cases

The table summarises the three cases for both search versions. The derivations follow the table.

Version	Best case	Average case	Worst case
S1 (sequential search)	The key is the first entry of the catalogue: 1 comparison, $\Theta(1)$.	Key present with probability p: $p(n + 1)/2 + (1 - p)n$ comparisons, $\Theta(n)$.	The key is the last entry, or is not in the catalogue: n comparisons, $\Theta(n)$.
S2 (binary search)	The key equals the middle entry of the whole index: 1 comparison, $\Theta(1)$.	About $\log_2 n - 1$ comparisons for a successful search, $\Theta(\log n)$.	The key is found only when one entry is left, or is absent: $\lfloor \log_2 n \rfloor + 1$ comparisons, $\Theta(\log n)$.

Search version S1 – average case with probability p

Step 1: The key is present with probability p. If it is present, it is equally likely to be at any of the n positions, so each position has probability p/n. Finding the key at position i takes i comparisons.

Step 2: The key is absent with probability $(1 - p)$. Then the whole catalogue is scanned, which takes n comparisons.

Step 3: The expected number of comparisons is the sum over all outcomes of (probability × comparisons):

$$C_{avg}(n, p) = \Sigma_{i=1}^{n} i \cdot (p/n) + n(1 - p)$$

Step 4: Use $\Sigma_{i=1}^{n} i = n(n + 1)/2$:

$$C_{avg}(n, p) = (p/n) \cdot n(n + 1)/2 + n(1 - p) = p(n + 1)/2 + n(1 - p)$$

Special cases: for p = 1 (always present) the average is (n + 1)/2, and for p = 0 (never present) it is n, which agrees with the standard results.

Step 5 – evaluation for n = 1,000,000 and p = 0.5:

$$C_{avg} = 0.5 \times (1{,}000{,}001)/2 + 1{,}000{,}000 \times (1 - 0.5)$$

$$C_{avg} = 250{,}000.25 + 500{,}000 = 750{,}000.25 \text{ comparisons}$$

At 10^8 operations per second, one search takes about 750,000.25 / 10^8 ≈ 0.0075 seconds (7.5 ms).

Search version S2 – best, average and worst cases

Best case: the key is exactly the middle entry, so it is found at the first comparison: 1 comparison.

Worst case: the key is found only after the range has been halved down to a single entry, or the key is absent. From T(n) = T(n/2) + 1, T(1) = 1 (part c), this is $\lfloor \log_2 n \rfloor + 1$ comparisons.

Average case (successful search): consider a binary decision tree with $n = 2^k - 1$ entries. Level l of the tree has 2^{l-1} entries, each found after l comparisons. The total number of comparisons over all n possible keys is $\Sigma_{l=1}^{k} l \cdot 2^{l-1} = (k - 1)2^k + 1$. Dividing by n gives about $k - 1 \approx \log_2 n - 1$ comparisons, which is Θ(log n). So the average is only about one comparison less than the worst case.

Assumption: This average-case calculation assumes that the search is successful and each of the n keys is equally likely to be selected. The perfect-tree formula is exact for $n = 2^k - 1$; for other n, the exact average depends on the shape of the binary-search decision tree, while the asymptotic bound remains Θ(log n).

Values for n = 1,000,000: best case = 1; worst case = $\lfloor 19.93 \rfloor + 1 = 20$; average ≈ 19.93 − 1 ≈ 19 comparisons. Compared with S1 (average 750,000.25 for p = 0.5 and worst 1,000,000), S2 needs about 50,000 times fewer comparisons in the worst case.

Part (f): Prove an Asymptotic Bound

Formal definition. f(n) = Θ(g(n)) if and only if there exist positive constants c_1, c_2 and n_0 such that $c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n)$ for all $n \geq n_0$.

To prove: $f(n) = 3n^2 + 10n \log_2 n + 50$ is $\Theta(n^2)$. Here $g(n) = n^2$.

Step 1 – lower bound ($c_1 \cdot n^2 \le f(n)$)

For $n \ge 1$, the terms $10n \log_2 n$ and 50 are non-negative. Dropping them can only decrease the value:

$$f(n) = 3n^2 + 10n \log_2 n + 50 \ge 3n^2$$

So $c_1 = 3$ works for all $n \ge 1$.

Step 2 – upper bound ($f(n) \le c_2 \cdot n^2$)

For every integer $n \ge 1$, $2^n \ge n$. Since $\log_2$ is increasing, taking base-2 logarithms gives $\log_2 n \le n$. Therefore

$$10n \log_2 n \le 10n \times n = 10n^2$$

Also, for $n \ge 1$, $n^2 \ge 1$, so

$$50 \le 50n^2$$

Adding the three terms:

$$f(n) = 3n^2 + 10n \log_2 n + 50 \le 3n^2 + 10n^2 + 50n^2 = 63n^2$$

So $c_2 = 63$ works for all $n \ge 1$.

Step 3 – conclusion

Choosing $c_1 = 3$, $c_2 = 63$ and $n_0 = 1$:

$$3n^2 \le f(n) \le 63n^2 \quad \text{for all } n \ge 1$$

By the definition, $f(n) = \Theta(n^2)$.

Numerical check. $n = 1$: $f = 3 + 0 + 50 = 53$ and $3 \le 53 \le 63$. $n = 10$: $f \approx 300 + 332.2 + 50 = 682.2$ and $300 \le 682.2 \le 6{,}300$. $n = 1{,}000$: $f \approx 3{,}000{,}000 + 99{,}658 + 50 = 3{,}099{,}708$ and $3{,}000{,}000 \le 3{,}099{,}708 \le 63{,}000{,}000$. All checks hold.

A tighter choice (optional). For $n \ge 64$ we have $10n \log_2 n + 50 \le n^2$ (at $n = 64$: $10 \times 64 \times 6 + 50 = 3{,}890 \le 4{,}096$, and the gap widens as n grows). Hence $3n^2 \le f(n) \le 4n^2$ for $n \ge 64$, so $c_1 = 3$, $c_2 = 4$, $n_0 = 64$ also works.

Proof that f(n) is not O(n)

Proof by contradiction. Suppose $f(n) = O(n)$. Then there exist constants $c > 0$ and n_0 such that $f(n) \le c \cdot n$ for all $n \ge n_0$.

Step 1: From the lower bound above, $f(n) \ge 3n^2$, so $3n^2 \le f(n) \le c \cdot n$ for all $n \ge n_0$.

Step 2: Dividing by 3n ($n > 0$) gives $n \leq c/3$ for all $n \geq n_0$.

Step 3: This is impossible, because n can be chosen larger than both n_0 and $c/3$ (for example $n = \max(n_0, \lceil c/3 \rceil + 1)$). This contradicts the assumption. Therefore f(n) is not O(n). ■

Alternative check with limits: $f(n)/n = 3n + 10 \log_2 n + 50/n \to \infty$ as $n \to \infty$, so f(n) grows strictly faster than n.

Meaning for ShopSphere. The report generator is a quadratic-time routine. The lower-order terms are practically irrelevant: at n = 1,000,000 the term $10n \log_2 n \approx 2 \times 10^8$ is only about 0.007% of the dominant term $3n^2 = 3 \times 10^{12}$.

Part (g): Scalability Decision

Budget. The nightly window is 10 minutes = 600 s. At 10^8 operations per second, at most 6×10^{10} basic operations can be executed in the window.

Operation counts used ($n = 1{,}000{,}000$, $\log_2 n \approx 19.93$):

- M1: $n(n-1)/2 = 10^6(10^6 - 1)/2 = 499{,}999{,}500{,}000 \approx 5.0 \times 10^{11}$ (part b).
- M2: $2^{\{\lceil \log_2 n \rceil\}} - 1 = 2^{20} - 1 = 1{,}048{,}575 \approx 1.05 \times 10^6$, because $2^{19} = 524{,}288 < 10^6 \leq 2^{20} = 1{,}048{,}576$ (part b).
- S2: $\lfloor \log_2 n \rfloor + 1 = 20$ comparisons per search (this is the cost of one search, not all nightly searches). (part c).
- M4 current: $2n^2 - n = 2 \times 10^{12} - 10^6 = 1{,}999{,}999{,}000{,}000 \approx 2.0 \times 10^{12}$ (part d).
- M4 redesign: $n \log_2 n + n \approx 19{,}931{,}569 + 1{,}000{,}000 \approx 2.09 \times 10^7$ (part d).
- M5 analytics: $n \cdot L(L+1)/2 + n$ with $L = 19.93$, which is $\approx 208{,}600{,}000 + 1{,}000{,}000 \approx 2.10 \times 10^8$ (part d). Without the factor 1/2, $n \log^2 n \approx 3.97 \times 10^8$; both values give a running time of a few seconds.
- M5 report generator: $f(n) = 3n^2 + 10n \log_2 n + 50 \approx 3 \times 10^{12} + 2 \times 10^8 \approx 3.0 \times 10^{12}$ (part f).

Running time at n = 1,000,000 (time = operations ÷ 10^8):

Module	Basic operations	Running time	Within the 10-minute window?
M1	$\approx 5.0 \times 10^{11}$	4,999.995 s ≈ 83.3 minutes	**No** (about 8.3 times over the budget)
M2	1,048,575 ≈ 1.05×10^{6}	≈ 0.0105 s (10.5 ms)	Yes
S2 (per search)	20	0.2 microseconds	Yes
M4 – current design	$\approx 2.0 \times 10^{12}$	≈ 20,000 s ≈ 5.6 hours	**No** (about 33 times over)
M4 – redesign	$\approx 2.09 \times 10^{7}$	≈ 0.21 s	Yes
M5 – analytics engine	$\approx 2.10 \times 10^{8}$	≈ 2.1 s	Yes
M5 – report generator	$\approx 3.0 \times 10^{12}$	≈ 30,002 s ≈ 8.3 hours	**No** (about 50 times over)

How the modules behave as the catalogue grows from 100,000 to 1,000,000 products:

Module	Time at n = 100,000	Time at n = 1,000,000	Growth factor
M1	≈ 50 s	≈ 5,000 s (83 min)	× 100
M2	131,071 operations ≈ 1.3 ms	≈ 10.5 ms	× 8 (steps with powers of 2; trend about × 10)
S2	17 comparisons	20 comparisons	+ 3 comparisons
M4 – current	≈ 200 s (3.3 min)	≈ 20,000 s (5.6 h)	× 100
M4 – redesign	≈ 0.018 s	≈ 0.21 s	× 12
M5 – analytics	≈ 0.15 s	≈ 2.1 s	× 14
M5 – report generator	≈ 300 s (5 min)	≈ 30,002 s (8.3 h)	× 100

Observation. Today (n = 100,000) every module still finishes inside the 10-minute window, which hides the problem. The three quadratic modules (M1, M4 current design and the report generator) are all running at 1 to 5 minutes today; they grow 100 times when the data grows 10 times and will therefore fail after the expansion.

Modules that fail the 10-minute window

M1 (83.3 minutes), M4 with its current design (5.6 hours) and the M5 report generator (8.3 hours) fail. All three are $\Theta(n^2)$, which is why they cannot absorb a 10-fold growth in n.

Recommendations

1. M1 – Duplicate-Listing Detector: must be redesigned. Replace the all-pairs comparison with a hash table keyed by (title, seller). Process the products in one pass: if the key is already in the table, call MarkDuplicate; otherwise insert the key. The expected time is $\Theta(n)$, which is about 10^6 operations (≈ 0.01 s), about 500,000 times fewer than the present 5×10^{11}. The price is $\Theta(n)$ extra memory. An alternative with no hash table is to sort the products by (seller, title) in $\Theta(n \log n)$ (about 2×10^7 operations, 0.2 s) and compare only neighbouring entries. Even better, a unique index on (title, seller) in the database can reject duplicates at upload time, so that the nightly job is not needed at all.

2. M4 – Recommendation Engine: adopt the proposed redesign. Removing the replicated partitions changes $T(n) = 4T(n/2) + n = \Theta(n^2)$ into $T(n) = 2T(n/2) + n = \Theta(n \log n)$. The work falls from about 2×10^{12} to 2.09×10^7 operations (about 95,000 times fewer) and the time from 5.6 hours to 0.21 s. Since replication was introduced for a reason (availability and locality), fault tolerance should be provided by other means, such as storage-level replication of the finished results.

3. M5 – Report generator: must be redesigned. Its cost is $\Theta(n^2)$ (part f) and it would take about 8.3 hours. The $3n^2$ term most likely comes from a nested pairwise pass over the records. It should be replaced by a single-pass aggregation using hash maps ($\Theta(n)$) or by sort-and-scan ($\Theta(n \log n)$), or by maintaining running totals incrementally during the day. Even if the constant factor 3 is kept, a $\Theta(n \log n)$ design needs only about 6×10^7 operations, which is less than one second.

4. M5 – Sales analytics engine: no redesign is required now, but an optimisation is advisable. It takes about 2.1 s, far below the window. Because $T(n) = 2T(n/2) + n \log n = \Theta(n \log^2 n)$, the cost is higher than necessary. If the records are sorted once (or the sorted halves are merged in linear time instead of re-sorting at each level), the recurrence becomes $T(n) = 2T(n/2) + n = \Theta(n \log n)$.

5. M2 and M3 (S2): keep as they are. M2 is $\Theta(n)$, about 10 ms at $n = 10^6$, and the team member's $\Theta(n \log n)$ claim should be corrected in the documentation. S2 is $\Theta(\log n)$, needing

only 20 comparisons. Product search should use S2 (a sorted index or a balanced-tree or B-tree index) instead of S1, because S1 needs up to 10^6 comparisons per query (about 10 ms) and 750,000 on average for p = 0.5, and this grows linearly with the catalogue.

Final conclusion

Before the catalogue expands to 1,000,000 products, three components must be redesigned: the duplicate detector M1, the recommendation engine M4 (use the no-replication design) and the nightly report generator of M5. After the redesign, all of them become $\Theta(n)$ or $\Theta(n \log n)$ and complete in a fraction of a second, while M2, S2 and the analytics engine already satisfy the nightly 10-minute window.

Code of Conduct

I J Rieona (192424268) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: J Rieona

Date: 12-10-2026